

Optional Proofs — Progress Update

Breakout Call, February 11, 2026

Francesco Risitano

Agenda

1. EngineAPI Refresher
2. Validator Bandwidth & Prover Interface
3. Protocol Data Types
4. Proof Engine
5. Process Block
6. Process Execution Proof
7. Request Proofs
8. Proof Gossip Protocol
9. Lighthouse Implementation
10. References

EngineAPI Refresher

```
class NewPayloadRequest(object):
    execution_payload: ExecutionPayload
    versioned_hashes: Sequence[VersionedHash]
    parent_beacon_block_root: Root
    execution_requests: ExecutionRequests

def new_payload(
    self: ExecutionEngine, new_payload_request: NewPayloadRequest
) -> bool:
    """
    Return ``True`` if and only if ``new_payload_request`` is valid`.
    """
    execution_payload = new_payload_request.execution_payload
    parent_beacon_block_root = new_payload_request.parent_beacon_block_root
    execution_requests = new_payload_request.execution_requests

    if b"" in execution_payload.transactions:
        return False

    # Assert that the block hash is valid with respect to the parent beacon block root, execution requests and execution payload.
    if not self.is_valid_block_hash(execution_payload, parent_beacon_block_root, execution_requests):
        return False

    # Assert that the versioned hashes are consistent with the transactions in the execution payload.
    if not self.is_valid_versioned_hashes(new_payload_request):
        return False

    # Assert that the execution payload is valid
    if not self.notify_new_payload(execution_payload, parent_beacon_block_root):
        return False

    return True
```

Validator Bandwidth & Prover Interface

```
def prove_payload(self: Prover, new_payload_request: NewPayloadRequest) -> Hash256:
    """
    Return ``Hash256`` of the new payload request tree hash root if and only if ``new_payload_request`` is valid.
    """
    execution_payload = new_payload_request.execution_payload
    parent_beacon_block_root = new_payload_request.parent_beacon_block_root
    execution_requests = new_payload_request.execution_requests

    if b"" in execution_payload.transactions:
        return False

    # Assert that the block hash is valid with respect to the parent beacon block root, execution requests and execution payload.
    assert self.is_valid_block_hash(execution_payload, parent_beacon_block_root, execution_requests)

    # Assert that the versioned hashes are consistent with the transactions in the execution payload.
    assert self.is_valid_versioned_hashes(new_payload_request)

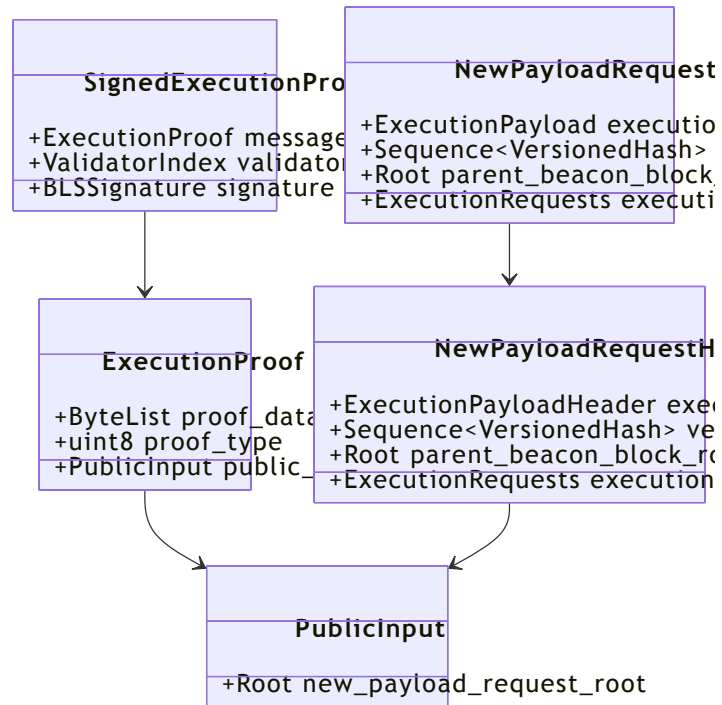
    # Assert that the execution payload is valid
    assert self.notify_new_payload(execution_payload, parent_beacon_block_root)

    new_payload_request_root = new_payload_request.hash_tree_root()

    return new_payload_request_root

def verify_payload(self: Verifier, proof_data: ByteList[MAX_PROOF_SIZE], new_payload_request_root: Hash256) -> bool:
    """
    Return ``True`` if and only if the new payload request tree hash root is valid.
    """
    # Assert that the new payload request tree hash root is valid
    return self.verify_proof(proof_data, new_payload_request_root)
```

Protocol Data Types



Proof Engine

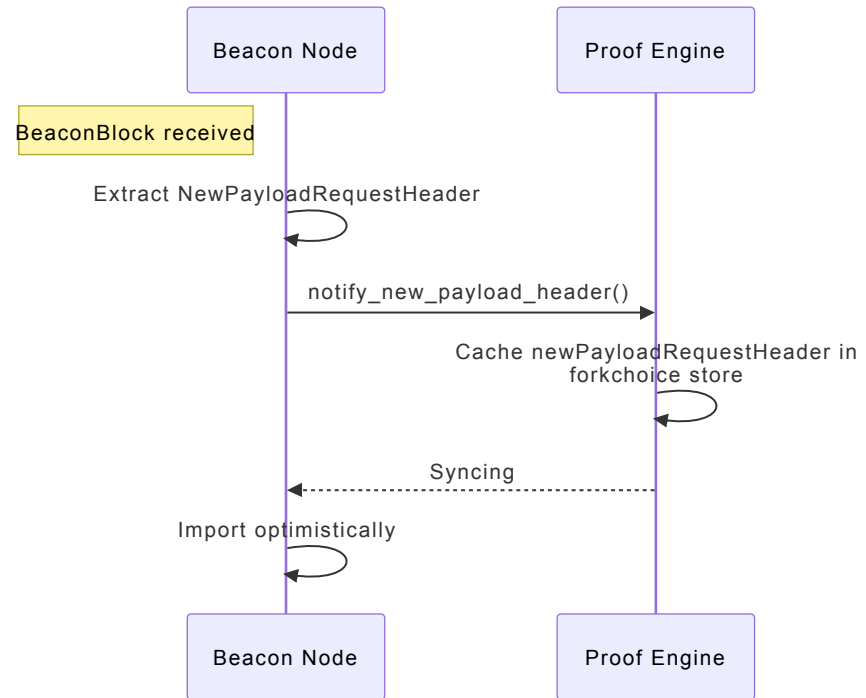
```
def verify_execution_proof(
    self: ProofEngine,
    execution_proof: ExecutionProof,
) -> bool:
    """
    Verify an execution proof.
    Return ``True`` if proof is valid.
    """
    ...

def notify_new_payload_header(
    self: ProofEngine,
    new_payload_request_header: NewPayloadRequestHeader,
):
    """
    Notify the proof engine of a new payload request header.
    """
    ...

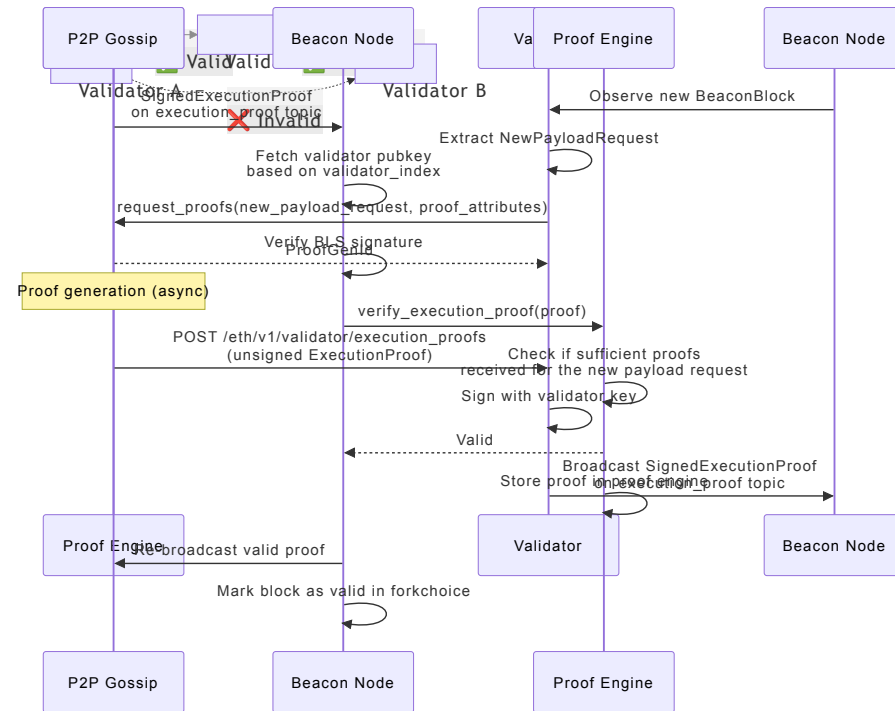
@dataclass
class ProofAttributes(object):
    proof_types: List[ProofType]

def request_proofs(
    self: ProofEngine,
    new_payload_request: NewPayloadRequest,
    proof_attributes: ProofAttributes,
) -> ProofGenId:
    """
    Request proof generation for a new payload request.
    """
    ...
```

Process Block



Process Execution Proof



Request Proofs

Proof Gossip Protocol

- Validators sign and broadcast proofs on execution_proof topic
- Nodes ban peers that send invalid proofs
- Nodes ban validators that sign invalid proofs

Lighthouse Implementation

- ☒ Proof engine service
- ☒ Execution proof gossip
- ☒ Execution proof signature verification
- ☒ Validator proof service
- ☒ Proof engine <-> Beacon chain integration
- ☒ Unit tests
- ☐ Historical proof sync
- ☐ Ban validators for invalid proofs
- ☐ Integration testing
- ☐ Kurtosis support

References

- [EIP-8025 Consensus Specs](#)
- [Lighthouse Implementation](#)

